

Module : Kotlin (M202)

TP N° 6

Année de Formation 2024/2025

Filière : Développement Mobile Groupe : DEVM 101 - DEVM 102 Niveau : 2ème année

Exercice 1 :

Créer un système de gestion de véhicules avec les exigences suivantes

- Interface **Vehicule** : Elle contiendra des propriétés et des méthodes que tout véhicule doit implémenter.
- Classe **VehiculeBase** : Une classe de base qui implémente l'interface **Vehicule**.
- Classes **Voiture** et **Camion** : Elles hériteront de la classe **VehiculeBase** et ajouteront des fonctionnalités spécifiques à chaque type de véhicule.

Camion : l'attribut `capaciteChargement` et la fonction `charger()`

Voiture : l'attribut `nombrePortes` et la fonction `klaxonner()`

Exercice 2 :

Créer un programme en Kotlin qui utilise une sealed class pour représenter et gérer différentes opérations mathématiques.

1. Définis une sealed class appelée **Operation** qui représente les opérations mathématiques suivantes :
 - **Addition**: pour ajouter deux nombres.
 - **Soustraction**: pour soustraire un nombre d'un autre.
 - **Multiplication**: pour multiplier deux nombres.
 - **Division**: pour diviser un nombre par un autre.
2. Chaque sous-classe d'**Operation** doit prendre deux nombres (a et b) en entrée.

3. Implémente une fonction **calculer**(operation: Operation): Double qui prend en entrée une instance de Operation et retourne le résultat de l'opération correspondante :
 - Pour **Addition**, retourne la somme de a et b.
 - Pour **Soustraction**, retourne la différence entre a et b.
 - Pour **Multiplication**, retourne le produit de a et b.
 - Pour **Division**, retourne le quotient de a divisé par b. Si b est égal à zéro, lance un message d'erreur pour éviter la division par zéro.
4. Dans la fonction main, crée des instances des différentes opérations (Addition, Soustraction, Multiplication, Division) et utilise la fonction calculer pour afficher le résultat de chaque opération.

Exercice 3 :

1. Créer une data class appelée Produit avec les propriétés suivantes :
 - **nom** : Le nom du produit (type String).
 - **prix** : Le prix du produit (type Double).
 - **quantite** : La quantité en stock du produit (type Int).
 - **description** : Une brève description du produit (type String).
2. Instancie un objet Produit dans la fonction **main**.
3. Affiche les détails du produit en utilisant la méthode **toString()** générée par défaut.
4. Modifie le prix ou la quantité du produit en utilisant la méthode **copy()**.
5. Compare deux produits pour vérifier s'ils sont identiques à l'aide de **equals()**.

Exercice 4 :

1. Créer une classe **Banque** qui représente une banque et qui contient une nested class appelée **CompteBancaire**.

La nested class **CompteBancaire** doit avoir les propriétés suivantes :

- **numeroCompte** : Le numéro du compte (type String).
 - **solde** : Le solde du compte (type Double).
2. Dans la classe **Banque**, ajoute une méthode **creerCompte**(numero: String, soldeInitial: Double) qui crée une nouvelle instance de CompteBancaire.
 3. Dans la nested class **CompteBancaire**, ajoute des méthodes pour :
 - Déposer de l'argent (**deposer**(montant: Double)).
 - Retirer de l'argent (**retirer**(montant: Double)).
 4. Dans la fonction main, crée une banque, ajoute un compte, et effectue des opérations sur ce compte.

Exercice 5 :

1. Créer une enum class appelée EtatCommande qui représente les différents états possibles d'une commande :
 - EN_COURS : La commande est en cours de traitement.
 - EXPEDIEE : La commande a été expédiée.
 - LIVREE : La commande a été livrée.
 - ANNULEE : La commande a été annulée.
2. Ajouter une méthode à l'enum class **EtatCommande** appelée **message()** qui retourne un message personnalisé en fonction de l'état de la commande.
3. Créer une classe Commande qui contient les propriétés suivantes :
 - **numeroCommande** : Le numéro de la commande (type String).
 - **etat** : L'état de la commande (type EtatCommande).
4. Ajoute une méthode dans la classe **Commande** pour changer l'état de la commande.
5. Dans la fonction main, crée une commande, affiche son état, modifie son état, et affiche le message correspondant.